Manual Uso

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect
Generado 2026-04-24

Manual de uso

> LegalDataPlatform — cómo levantar y operar la plataforma en tu máquina en 9 pasos.

Introducción

En los próximos minutos vas a levantar una plataforma de datos completa en tu máquina local, ingerir datos sintéticos, procesarlos por un pipeline ETL/ELT real y consultar los resultados.

Todo corre contra **MinIO** (S3 local), sin necesidad de credenciales AWS ni tarjeta de crédito.

Prerequisitos

- Python 3.11 o superior
- Docker Desktop (incluye Docker Compose)
- Git
- ~2 GB de RAM libre para los contenedores
- ~3 GB de disco libre para imágenes y datos

Tiempo estimado: 10–15 minutos la primera vez; menos de 1 minuto en runs posteriores.

Paso 1 de 9 — Ubícate en el proyecto

Abre tu terminal (PowerShell, cmd, Git Bash o WSL) y navega a la carpeta del proyecto:

```
cd E:\Proyectos\LegalDataPlatform
```

Verifica la estructura esperada:

```
ls        # Mac/Linux/Git Bash
dir       # Windows cmd/PowerShell
```

Deberías ver:

Carpetas: `src/`, `infra/`, `tests/`, `docs/`, `scripts/`

Archivos clave: `docker-compose.yml`, `pyproject.toml`, `Makefile`, `alembic.ini`, `.env.example`

Paso 2 de 9 — Configura las variables de entorno

Copia el archivo de ejemplo como punto de partida:

```
cp .env.example .env
```

El `.env.example` ya trae valores funcionales para desarrollo local:

```
# PostgreSQL
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=legaldata
POSTGRES_USER=ldp_admin
POSTGRES_PASSWORD=ldp_dev_password

# S3 local (MinIO)
AWS_S3_ENDPOINT=http://localhost:9000
AWS_ACCESS_KEY_ID=minioadmin
AWS_SECRET_ACCESS_KEY=minioadmin123

# Prefect
PREFECT_API_URL=http://localhost:4200/api
```

> **Aviso:** nunca commitees un `.env` con credenciales reales. El archivo ya está en `.gitignore`. En producción las credenciales vienen de AWS Secrets Manager vía la task definition de ECS o el IRSA del pod de Kubernetes.

Paso 3 de 9 — Levanta el stack local con Docker

Un solo comando arranca los 6 servicios:

```
docker compose up -d
```

Verifica que todos estén `running`:

```
docker compose ps

NAME              STATUS
ldp_postgres      Up (healthy)
ldp_pgbouncer     Up
ldp_minio         Up (healthy)
ldp_prefect       Up
ldp_prometheus    Up
ldp_grafana       Up
```

Si algún servicio aparece en `Restarting`, revisa sus logs:

```
docker compose logs postgres
docker compose logs -f prefect    # -f hace "follow" (tail -f)
```

> **Primera vez:** Docker descarga las imágenes (~2 GB total). Puede tardar 2–5 minutos. Las siguientes veces `up -d` es instantáneo.

Paso 4 de 9 — Instala las dependencias Python

Se recomienda un virtualenv aislado:

```
python -m venv .venv

source .venv/bin/activate      # Mac / Linux / Git Bash
.venv\Scripts\activate         # Windows PowerShell / cmd

pip install -e ".[dev]"
```

Flags:
- `-e` (editable): los cambios al código se reflejan sin reinstalar.
- `[dev]`: incluye tooling de desarrollo (pytest, mypy, ruff, moto, faker).

Se instalan ~40 librerías. Las más importantes:

```
pandas, polars              # procesamiento de datos
sqlalchemy[asyncio]         # ORM async
asyncpg                     # driver PostgreSQL async
pydantic                    # validación
prefect                     # orquestación
great-expectations          # data quality
boto3, aioboto3             # AWS SDK
structlog, prometheus-client # observabilidad
pytest, moto, faker         # testing
```

Paso 5 de 9 — Aplica las migraciones de schema

La base de datos está vacía. Alembic crea toda la estructura:

```
alembic upgrade head
```

Esto ejecuta las 2 migraciones en orden:

0001_initial_schema

Crea extensiones (`uuid-ossp` , `pgcrypto` , `pg_trgm` , `btree_gin`), las 9 tablas principales con sus constraints (FK, CHECK, UNIQUE), índices base y tablas de staging UNLOGGED.

0002_partitions_and_mviews

Define la función helper `create_monthly_partition()` , genera particiones desde hace 6 meses hasta +12 meses, y crea la materialized view `mv_monthly_revenue_per_counterparty` .

Verifica el resultado:

```
psql -h localhost -U ldp_admin -d legaldata -c "\dt"
psql -h localhost -U ldp_admin -d legaldata -c "\d+ transactions"
```

Paso 6 de 9 — Genera datos sintéticos

El script de seed usa Faker para producir datasets realistas:

```
python scripts/seed_data.py
```

Resultado en `data/samples/` :

Archivo	Filas	Contenido
`legal_documents.csv`	5,000	Sentencias, regulaciones, actas. Incluye ~50 filas inválidas intencionadas.
`counterparties.csv`	500	Contrapartes con `risk_score` , `country_code` , `tax_id` .
`contracts.csv`	1,500	Contratos con vigencias, moneda y valor total.
`transactions.csv`	20,000	Facturas, pagos y refunds con referencias únicas.

> **Diseño intencional:** el seed incluye ~1% de registros malformados. Esto permite ver en vivo el pipeline rechazándolos a *quarantine* y validar que la observabilidad de DQ funciona.

Paso 7 de 9 — Ejecuta el pipeline end-to-end

Corre el flow principal:

```
make pipeline
# equivalentemente:
python -m src.pipelines.orchestration.legal_ingestion_flow
```

Output esperado (resumido):

```
[INFO] csv_extractor.start path=data/samples/legal_documents.csv
[INFO] csv_extractor.batch batch_id= ...  size=5000 offset=0
[INFO] s3_loader.written dataset=legal_documents rows=5000
[INFO] normalizer.rejections schema=LegalDocumentSchema
       rejected=50 valid=4950 rate=0.01
[INFO] dq.checks_passed suite=legal_ingestion total=6
[INFO] pg_loader.upserted table=legal_documents rows=4950
       conflict=source_system,source_id,document_date
[INFO] Flow summary: {"extracted":5000,"valid":4950,
       "rejected":50,"loaded":4950}
```

Métricas del run: 5,000 registros procesados, 50 rechazados y enviados a quarantine, 4,950 escritos a Bronze (S3) + Silver (PostgreSQL). Materialized views refrescadas. Duración total < 60 segundos.

Para el pipeline comercial (counterparties + transactions):

```
python -m src.pipelines.orchestration.commercial_ingestion_flow
```

Paso 8 de 9 — Inspecciona los resultados

UIs web

UI	URL	Credenciales
Prefect	http://localhost:4200	— (sin auth local)
MinIO Console	http://localhost:9001	`minioadmin` / `minioadmin123`
Prometheus	http://localhost:9090	—
Grafana	http://localhost:3000	`admin` / `admin`

Consulta directa contra PostgreSQL

```
psql -h localhost -U ldp_admin -d legaldata
```

```
SELECT COUNT(*), MIN(document_date), MAX(document_date)
  FROM legal_documents;

SELECT * FROM mv_monthly_revenue_per_counterparty LIMIT 10;

SELECT queryid, calls, mean_exec_time, query
  FROM pg_stat_statements
  ORDER BY mean_exec_time DESC LIMIT 5;
```

Paso 9 de 9 — Tests, benchmarks y calidad

Suite de tests

```
make test
```

Corre ~30 tests unitarios (pytest-asyncio) + tests de integración contra S3 mockeado con moto. Los tests cubren:

- Schemas Pydantic: validación de campos, rangos, normalización
- Rule engine: cada tipo de regla YAML (`not_null` , `unique` , `regex` , `in_set` , `range`)
- Helpers de enrichment: row hash determinista, risk tier buckets
- Normalizers: separación valid/rejected
- S3 handler: put/list/delete contra moto

Benchmarks

```
make benchmark
```

```
═══ count_transactions_last_30d ═══
p50: 12.34 ms   p95: 18.90 ms   p99: 22.15 ms
Cache hit: 99.87%

═══ top_counterparties_by_revenue ═══
p50: 45.67 ms   p95: 72.13 ms   p99: 89.40 ms
Cache hit: 95.20%

═══ mv_monthly_revenue_scan ═══
p50: 2.15 ms    p95: 3.80 ms    p99: 5.42 ms
```

Linting y formato

```
make lint          # ruff (lint) + mypy (type check)
make format        # ruff format + auto-fix
```

Cheatsheet de comandos

Operación	Comando
Estado de servicios	`docker compose ps`
Logs de un servicio	`docker compose logs -f postgres`
Reiniciar un servicio	`docker compose restart prefect`
Apagar stack (conserva volúmenes)	`docker compose down`
Apagar y reset total	`docker compose down -v`
Pipeline legal	`make pipeline`
Pipeline comercial	`python -m src.pipelines.orchestration.commercial_ingestion_flow`
Registrar deployments Prefect	`python -m src.pipelines.orchestration.deployments`
Nueva migración	`alembic revision --autogenerate -m "descripción"`
Aplicar migraciones	`alembic upgrade head`
Rollback última migración	`alembic downgrade -1`
Tests	`make test`
Benchmarks	`make benchmark`
Regenerar PDFs	`python scripts/generate_pdfs.py`
Terraform plan	`cd infra/terraform && terraform plan -out tfplan`
Terraform apply	`terraform apply tfplan`

Troubleshooting — problemas comunes

"port 5432 already in use"

Otro PostgreSQL local está corriendo. Detenlo (`pg_ctl stop` , service manager o Docker Desktop) o cambia el puerto en `docker-compose.yml` a 5433.

"connection refused" hacia postgres

El healthcheck tarda unos segundos. Espera 10s después de `docker compose up` . Confirma con `docker compose logs postgres` que aparezca "*database system is ready to accept connections*".

"ModuleNotFoundError: No module named 'src'"

Falta instalar en modo editable. Ejecuta `pip install -e ".[dev]"` desde la raíz del proyecto, dentro del virtualenv activo.

Pipeline rechaza el 100% de las filas

Muy probable que falten migraciones o el seed. Ejecuta en orden: `alembic upgrade head` → `python scripts/seed_data.py` → `make pipeline`.

"InvalidAccessKeyId" contra S3 local

MinIO no arrancó correctamente o el `.env` tiene credenciales distintas a las del `docker-compose.yml`. Verifica ambos y reinicia: `docker compose restart minio`.

Prefect UI no carga

Revisa `docker compose logs prefect`. Si dice "port 4200 in use", detén el proceso anterior. Si dice "waiting for database", espera 30s adicionales para que Prefect inicialice su SQLite interna.

"asyncpg.exceptions.PreparedStatementError"

Conectaste vía PgBouncer sin deshabilitar el statement cache. Usa `direct_engine` o configura `connect_args={"statement_cache_size": 0}` en el engine.

Listo

Si llegaste hasta aquí, tienes un stack de datos profesional corriendo localmente: ingesta, validación, normalización, almacenamiento y observabilidad.

Para profundizar

Consulta los PDFs en `docs/pdf/`:

- **WALKTHROUGH.pdf** — explicación técnica completa (6 ADRs, decisiones de diseño)
- **ARCHITECTURE.pdf** — diagramas, componentes y patrones aplicados
- **PIPELINES.pdf** — extractors, transformers, loaders, orquestación
- **POSTGRESQL_OPTIMIZATION.pdf** — tuning, particionado, índices, benchmarks
- **DATA_QUALITY.pdf** — las 3 líneas de defensa (Pydantic, YAML, GE)
- **AWS_DEPLOYMENT.pdf** — Terraform, costos mensuales, runbooks de DR